

PipeNet description

- To: cypherpunks@toad.com
 - Subject: PipeNet description
 - From: Wei Dai <weidai@eskimo.com>
 - Date: Mon, 19 Jan 1998 23:53:25 -0800
 - Sender: owner-cypherpunks@Algebra.COM
-

I just got a couple of requests for information about PipeNet and I realized that I never did make my final description of it public. The attached informal write-up was done shortly after Crypto '96 as a result of discussions with Hal Finney, David Wagner, and Eric Hughes. I haven't worked on it since then, mostly because the Onion Routing project is doing similar work and appears to be much better funded (I did send them a copy of this write-up per their request).

The Model

Consider a network of processors which send messages to each other asynchronously. We assume that each node is identified by its public key, and that links between nodes are secure. The adversary may control a fixed subset of the nodes. However all messages between any two honest nodes are confidential and always arrive (unmodified) in the order sent (this can be achieved with a standard transport level security protocol).

The Protocol

The protocol is based on the idea of virtual link encryption. After an anonymous connection is established, the originating node sends a constant stream of (constant size) packets to a second node at a fixed rate. The second node shuffles the packets it receives during a time unit and forwards them in random order to others.

A connection is a path in the network. The first node in the path is the caller, and the last node is the receiver. The rest are called switches. Anonymity in this scheme is asymmetric - the caller is anonymous, but not the receiver. Each node in the path shares a key with the caller and knows the nodes immediately in front of and behind it. Every pair of adjacent nodes shares a link id.

Each switch in the path therefore has a key and two associated ids. Call the id that it shares with the previous node (the one closer to the caller) type A, and call the other id type B. For each id, the switch expects exactly one packet tagged with that id in each time unit. When it receives a packet tagged with a type A id, it decrypts that packet with the associated key, tags it with the corresponding type B id, and forwards it to the next node. When it receives a packet tagged with a type B id, it encrypts the packet with the associated key, tags it with the corresponding type A id, and forwards it to the previous node. The forwarding is always done at the end of the time unit, and the order in which packets are sent is random.

For each packet going from the caller to the receiver, the caller must multi-encrypt it with the keys it shares with each of the nodes in the path, starting with the receiver. It then sends the packet to the first switch tagged with the appropriate link id. Each switch will strip a layer of encryption and forward the packet to the next node. For a packet going from the receiver to the caller, the receiver encrypts it with the key it shares with the caller and sends it to the last switch

Each switch will add a layer of encryption and forward the packet to the previous node.

To establish a connection, the caller (N0) performs the following steps:

1. Select a node (N1) at random and establish a key (K1) and link id (S1) with N1.
2. Select another node (N2) at random.
3. Request that N1 establish a link id (S2) with N2.
4. Establish a key (K2) with N2 through N1.
5. Request that N1 use K1 to decrypt all messages tagged with S1, tag the decrypted message with S2 and forward them to N2. Also request that N1 use K1 to encrypt all messages tagged with S2, tag the encrypted messages with S1 and forward them to N0.
6. Repeat steps 2 to 4 l-2 times. (Name the i-th node, key, and id Ni, Ki, and Si respectively.)
7. Repeat steps 2 to 4 a final time, but with the receiver node (Nl) instead of a random node.

Timing

We assumed earlier that communications links between honest nodes are vulnerable to delay attacks. Therefore we must ensure that link delays do not reveal traffic information. Each node expects one packet from each link id in each time unit. Extra packets are queued for processing in later time units. However, if it does not receive a packet for a link id in a particular time unit, it stops normal processing of packets for that time unit and queues all packets. This ensures that any delay is propagated through the entire network and cannot be used to trace a particular connection.

The process of making and breaking connections must also not leak information. This can be done by using a protocol analogous to mix-net. Link forming/destroying requests are queued and performed in batches in a way similar to queuing and mixing of e-mail in a mix-net.

-
- Prev by Date: [Re: RC1 and RC3 ??](#)
 - Next by Date: [Re: British Ministers Adopt Unbreakable Crypto](#)
 - Prev by thread: [Re: \\$50 encryption policy contest](#)
 - Next by thread: [API Specification for the AES](#)
 - Index(es):
 - [Date](#)
 - [Thread](#)